

Predicting Chess Position Evaluation from FEN using Feature Engineering and Classical Regression Models

Ferran Villarta^{a,1} and Natan Sisoev^{b,2}

^a1704051
^b1706198

Abstract—This report presents a machine-learning framework for predicting chess position evaluations in centipawns from FEN notation. The approach relies on extensive feature engineering to extract structured representations of positions and employs classical regression models to capture the relative evaluation between positions. Spearman rank correlation is used as the primary evaluation metric to measure the preservation of ranking order between positions.

Keywords—Chess centipawn evaluation, Machine Learning, Feature Engineering, FEN, Regression

Contents

1	Introduction	1
2	Dataset	1
3	Feature Engineering	2
3.1	Correlation Analysis	2
3.2	Feature Importance	2
4	Model Selection	3
5	Hyperparameter Optimization	4
5.1	Ridge Regression	4
5.2	XGBRegressor	4
5.3	KNeighborsRegressor	4
5.4	Final Remarks	4
6	Final Testing	4
6.1	Comparison of Optimized Models	4
6.2	Generalization Across Independent Datasets	5
7	Further Study	5
8	Code Usage	5
	References	5

1. Introduction

Chess positions can be assigned a numeric evaluation, known as a *centipawn score*, which quantifies the relative advantage of one side over the other. These evaluations provide a measure of position strength: a positive score indicates a White advantage, a negative score indicates a Black advantage, and approximately 100 centipawns correspond to the value of a single pawn. It is important to note that no exact function exists to determine the absolute value of a position; all centipawn evaluations are heuristic approximations.

Modern chess engines operate by exploring possible moves using **minimax-based algorithms**, which recursively evaluate the consequences of sequences of moves. At the leaves of the search tree, positions are scored using an *evaluation function*, which is a critical component of engine decision-making.

This project focuses on approximating such evaluation functions using **classical machine learning models**, deliberately excluding neural networks. Since these models cannot process raw board representations directly, the challenge lies in *feature engineering*: constructing a set of informative features from FEN strings that encapsulate the strategic, tactical, and material characteristics of positions.

To assess model performance, we employ the **Spearman rank correlation coefficient** as the primary metric, defined as

$$\rho = 1 - \frac{6 \sum_{i=1}^n d_i^2}{n(n^2 - 1)}, \quad d_i = \text{rank}(y_i) - \text{rank}(\hat{y}_i),$$

where y_i and $\hat{y}_i$ denote the true and predicted evaluations, respectively. This metric emphasizes the preservation of relative ordering between positions, which aligns with the requirements of minimax search: it is more important for an engine to rank positions correctly than to predict their absolute scores. Spearman correlation was used consistently for cross-validation and hyperparameter optimization to select the best-performing models.

Overall, this study is motivated by the need for *interpretable and computationally efficient evaluation functions* for classical chess engines. By leveraging feature engineering and regression models, we aim to approximate centipawn evaluations accurately, enabling engines to make informed decisions without relying on large neural networks or extensive handcrafted heuristics. The project demonstrates that careful feature design, coupled with rigorous evaluation, can yield practical and effective position evaluation models suitable for classical search-based chess engines.

2. Dataset

The dataset comprises approximately 13 million chess positions extracted from real games, totaling roughly 1 GB in raw format. Each position is represented by its **FEN** [3] string and an associated **centipawn evaluation** computed by the Stockfish 11 engine at a search depth of 22 [2].

Exploratory analysis of the raw evaluations indicates that the distribution is approximately centered around zero, reflecting that most positions are balanced, while a slight bias in favor of White is observed, consistent with the first-move advantage inherent to chess. Extreme values exceeding ± 1000 centipawns are rare, representing less than 5% of the dataset, and the standard deviation of the evaluations is roughly 800 centipawns.

During preprocessing, positions representing forced checkmates, identified by the `#` notation, were removed. Although these positions correspond to theoretically decisive outcomes, they constitute less than 1.5% of the data and were excluded to prevent distortion in model training; they could be considered for further study. Similarly, positions with absolute centipawn evaluations exceeding 1000 were discarded, as such extreme cases typically correspond to imminent checkmates or highly unbalanced positions, which could influence model learning and reduce generalization performance. Figure 1 illustrates the resulting distribution of evaluations after cleaning.

The dataset contains consecutive positions from the same game, typically spanning approximately 100 moves. Upon inspection, it is evident that the evaluations of these positions are highly correlated, indicating that the data cannot be assumed to be entirely independent.

As shown in Figure 2, the evaluations exhibit a clear periodic pattern, reflecting the natural progression of a single game. This observation highlights the potential for data leakage if consecutive positions were randomly assigned to training and test sets. To mitigate this issue, grouped sampling and cross-validation were implemented at the game level, ensuring that all positions from a given

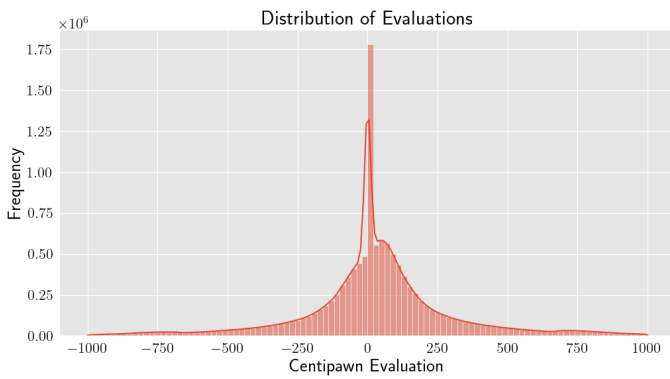


Figure 1. Distribution of centipawn evaluations in the dataset after cleaning. Most positions remain centered around zero, with extreme values removed.



Figure 2. Centipawn evaluations over consecutive positions in the dataset, illustrating the periodic pattern of highly correlated positions.

game reside within the same fold. This procedure preserves the independence of the training and test sets and allows for more reliable estimates of model performance.

3. Feature Engineering

We designed 81 features organized into seven conceptual categories—material, pawn structure, king safety, mobility, board control, tactical relations, and game state—each capturing complementary aspects of the game. For instance, without board control metrics, a side may appear materially strong but have pieces confined to ineffective squares, obscuring the true evaluation. Similarly, pawn structure features allow the model to recognize long-term weaknesses or potential breakthroughs that material counts alone cannot capture.

All features were computed once and stored in optimized formats (e.g., downcasted numeric arrays and serialized pickled files) to minimize memory usage and accelerate future usage. Preprocessed datasets can be directly loaded using:

```
dm: DataManager = load_dataset("one")
```

Features were computed separately for White and Black rather than as differences. This design choice reflects that absolute material matters: having 16 White pieces versus 14 Black pieces represents a fundamentally different position than 3 White versus 1 Black, even though both yield a +2 piece difference. This symmetry preserves information about the absolute state of the board, which is critical for accurate evaluation.

- **Piece Information** (16 features): Total piece counts per side (White_Pieces, Black_Pieces), total material value (White_Piece_Value, Black_Piece_Value), central piece presence (White_Central_Pieces, Black_Central_Pieces), and individual piece type counts for each side (White_Pawn through Black_Queen). These features capture absolute material strength and centralization, directly influencing tactical and strategic potential.
- **Pawn Structure** (8 features): Structural weaknesses and strengths including isolated pawns

(Isolated_Pawns_White/Black), doubled pawns (Doubled_Pawns_White/Black), passed pawns (Passed_Pawns_White/Black), and pawn island fragmentation (Pawn_Islands_White/Black). This set identifies long-term positional vulnerabilities and potential breakthroughs that material counts alone cannot reveal.

- **King Safety** (15 features): King security metrics including castling status (King_Castled_White/Black), pawn shield quality (King_Pawns_White/Black, King_Front_Pawns_White/Black), castling rights availability (King_Castling_Rights_White/Black), distance to back rank (King_Distance_to_Last_Rank_White/Black), check status (King_In_Check_White/Black), attacked neighboring squares (King_Attacked_Neighbours_White/Black), and board open files (Open_Files). These features quantify immediate and potential threats to king safety, which strongly influence evaluation.
- **Mobility** (12 features): Legal move counts for each piece type per side, measuring tactical flexibility: Mobility_King/Queen/Rook/Bishop/Knight/Pawn_White and corresponding Black mobility features. High mobility indicates greater control and tactical options.
- **Attack** (10 features): Tactical pressure metrics including threats created (Threats_Created_White/Black), hanging pieces and their point values (Hanging_Pieces_White/Black, Hanging_Points_White/Black), and undefended pieces with their values (Undefended_Pieces_White/Black, Undefended_Points_White/Black). These capture short-term tactical opportunities and vulnerabilities.
- **Board Control** (15 features): Strategic positioning indicators including central square control (Central_Squares_Control_White/Black), total controlled squares (Controlled_Squares_White/Black), open and semi-open file counts (Open_Columns, SemiOpen_Columns_White/Black), rook placement on open files (Rook_on_Open_Column_White/Black), protected advanced pawns (Protected_Advanced_Pawn_White/Black), and piece alignment (Rook_Queen_Aligned_White/Black, Rooks_Aligned_White/Black). These quantify board influence beyond raw material.
- **Game Information** (5 features): Meta-features capturing game state: halfmove clock since last capture or pawn move (Halfmove_Clock), total move count (Fullmove_Number), estimated game phase (Phase), active player (Side_To_Move_White), and en passant opportunity (Has_En_Passant). These features provide context necessary for evaluation across different game phases.

3.1. Correlation Analysis

Correlation analysis reveals that no single feature dominates the evaluation (illustrated in Figure 3). The strongest feature-target correlations reach only approximately $|r| \approx 0.15$, indicating that the evaluation emerges from complex, non-linear interactions among multiple features.

The intercorrelation matrix of these top 10 features, displayed in the accompanying heatmap (Figure 4), demonstrates moderate relationships between certain positional metrics, but overall confirms that features provide largely complementary information rather than redundant signals.

3.2. Feature Importance

To assess the practical contribution of each feature to model predictions, we evaluated feature importance using our final optimized

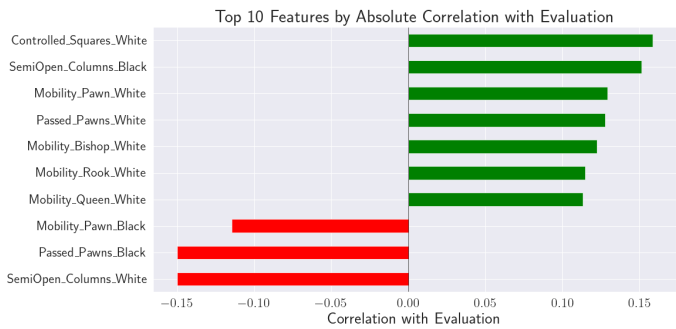


Figure 3. Top 10 features most correlated with evaluation.

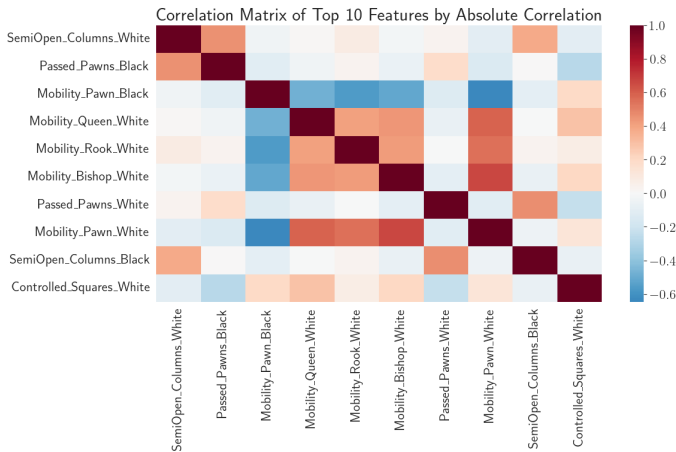


Figure 4. Inter-correlation matrix of the top 10 features most correlated with evaluation.

XGBoost regressor and the gain metric. While all 81 features were retained for modeling due to their complementary nature, the top 10 features by gain are summarized in Table 1. These include both tactical and strategic metrics such as semi-open columns, material values, and controlled squares, highlighting that evaluation depends on a combination of spatial control, piece activity, and potential tactical opportunities rather than any single dominant factor.

Table 1. Top 10 features by XGBoost gain.

Feature	Importance	Rescaled	Fraction
SemiOpen_Columns_White	15,572,475	1.000	0.0663
SemiOpen_Columns_Black	13,316,659	0.855	0.0567
Black_Piece_Value	12,698,469	0.815	0.0541
White_Piece_Value	11,768,498	0.756	0.0501
Controlled_Squares_White	11,137,458	0.715	0.0474
Controlled_Squares_Black	10,168,657	0.653	0.0433
Mobility_Pawn_White	7,333,354	0.471	0.0312
White_Queen	6,394,361	0.411	0.0272
Black_Queen	6,174,374	0.396	0.0263
Mobility_Pawn_Black	6,111,196	0.392	0.0260

Importance is rescaled relative to the highest value; fraction indicates proportion of total gain contributed.

It is worth noting that the features highlighted here by XGBoost gain are not identical to those appearing in Figure 3. In the importance-based ranking, both White and Black versions of key features often appear, reflecting that the model accounts for asymmetries in positional dynamics: for example, SemiOpen_Columns_White and SemiOpen_Columns_Black both rank highly, consistent with the idea that open or semi-open files influence each side’s potential activity. Similarly, material-related

features for both colors appear near the top, indicating that absolute piece value contributes significantly to evaluation beyond simple linear correlation. Overall, the presence of both White and Black variants aligns with chess intuition, as the model must consider each side’s assets and vulnerabilities to accurately predict position strength.

In summary, correlation analysis and feature importance jointly indicate that evaluation relies on subtle interactions among multiple aspects of the position. Retaining all 81 features allows the model to leverage these complex relationships, ensuring robust and interpretable predictions across diverse game scenarios.

4. Model Selection

A broad set of regression models was evaluated to identify architectures compatible with fast evaluation and consistent performance under Spearman rank correlation. Since a practical chess engine must compute thousands of node evaluations per second, both statistical accuracy and computational efficiency were treated as primary criteria. Methods with slow prediction times are unsuitable regardless of their raw metric values.

We grouped candidates into three families: linear models, decision-tree-based models, and a heterogeneous “other” category that includes instance-based, kernel-based, and neural approaches. Each model was tested with default hyperparameters on the one dataset (100k random positions). Fit time, prediction time, and mean Spearman rank correlation across folds were recorded.

Table 2. Comparison of representative models.

Model	Fit (s)	Prediction (s)	Spearman
Ridge Regression	0.148	0.012	0.520
XGBRegressor	4.140	0.103	0.595
KNeighborsRegressor	0.075	17.188	0.373
SVR	1107.069	404.421	0.457
MLPRegressor	636.283	0.286	0.614

Times correspond to mean cross-validated values. Only a subset of models is shown: Ridge represents linear approaches, XGBRegressor represents tree-based methods, and the remaining entries illustrate non-linear or instance-based alternatives.

Across linear models, Ridge Regression offered the best balance of speed and stability. Its performance was indistinguishable from unregularized linear regression, and substantially better than Lasso or ElasticNet in terms of computational cost. Because feature dimensionality is high and mild regularization prevents overfitting without sacrificing accuracy, Ridge was selected as the linear baseline for further tuning.

Tree-based models consistently achieved stronger rank correlations, with XGBRegressor and LGBMRegressor clearly outperforming Random Forests, Extra Trees, and classical boosting methods at a fraction of their computational cost. Among these, XGBRegressor provided the fastest prediction speed while maintaining the highest metric score, making it the most appropriate non-linear model for use in a time-constrained evaluation loop.

The remaining methods showed mixed suitability. KNeighborsRegressor performed poorly and, more importantly, exhibited prediction times several orders of magnitude too large for deployment in any minimax or alpha-beta search. Because instance-based methods scale their query cost linearly with dataset size, they become strictly impractical as training data grows. SVR suffered from extreme computational cost during both fitting and prediction and was therefore discarded. The MLPRegressor achieved the highest Spearman correlation overall, but required extensive training time and failed to converge under default iteration limits. Since the project focuses on classical, interpretable, and fast models rather than neural architectures, it was not pursued further.

In summary, the final model shortlist consists of Ridge Regression for the linear family and XGBRegressor for the decision-tree-based family. KNeighborsRegressor was initially retained for hyperparameter exploration. These selections balance predictive quality with the strict computational constraints of real-time chess evaluation.

5. Hyperparameter Optimization

The next step was therefore to perform detailed hyperparameter optimization for each of these three models using cross-validation and structured parameter grids.

5.1. Ridge Regression

For Ridge Regression, we optimized the regularization strength α across several logarithmic scales. As shown in Fig. 5, the score is remarkably stable across six orders of magnitude, confirming that the model is not highly sensitive to this parameter. A finer grid around the observed plateau revealed a mild peak near $\alpha = 100$, which we therefore adopt as the final setting. This value provides a slight gain in test stability without penalizing training performance.

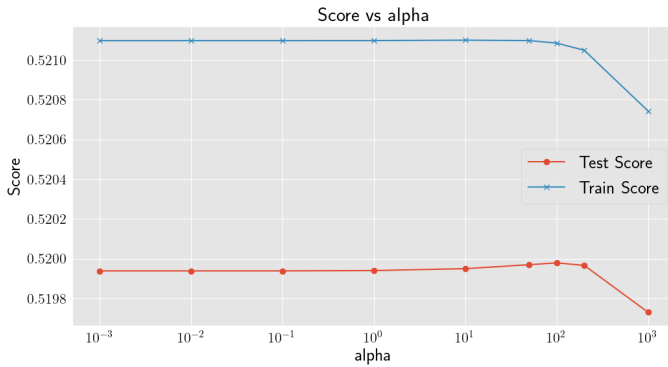


Figure 5. Ridge Regression α optimization score plot.

5.2. XGBRegressor

The XGBRegressor has a richer hyperparameter surface, so we explored combinations of learning rate, number of estimators, and maximum depth. The initial grid search included a wide range of values without a lot of resolution, which upon later iteration was more finely tuned. The best results consistently appeared at high estimator counts and learning rates around 0.1.

To refine this, we fixed the maximum depth to 7—a depth that balances complex pattern retention with resistance to noise memorization (because of the large dataset)—and performed a second, finer grid search. This yielded a clear improvement for a learning rate of 0.05 together with a high estimator count of 3000 (see Fig. 6). These settings maximize test performance while remaining stable under cross-validation.

5.3. KNeighborsRegressor

The KNN model was placed inside a pipeline including a StandardScaler, allowing distance computations to operate on normalized features. We evaluated a broad range of neighborhood sizes k , producing the curve shown in Fig. 7. The training score unsurprisingly starts at $\rho = 1$ for $k = 1$, but quickly collapses as k increases. The test score improves initially, but plateaus in the region $k \in [50, 150]$, indicating that local averaging stabilizes but does not lead to meaningful generalization increases.

Although we identified an approximate optimum around $k = 50$, further analysis revealed that the method's structural limitations outweighed any benefits of fine-tuning. Specifically:

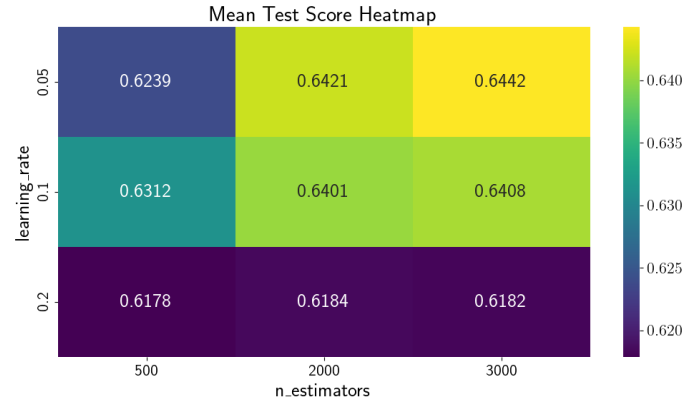


Figure 6. XGBRegressor $n_{\text{estimators}}$ and η optimization score heatmap.

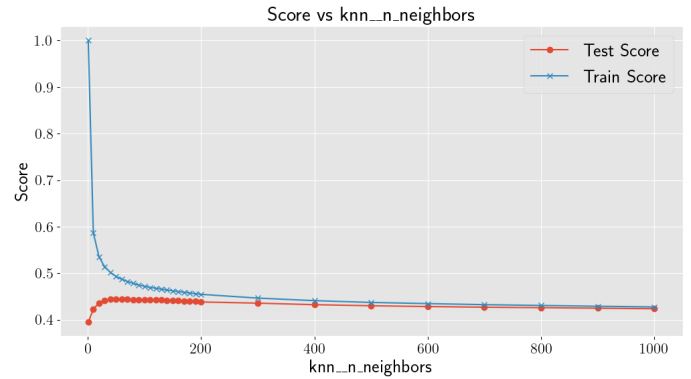


Figure 7. XGBRegressor $n_{\text{estimators}}$ and η optimization score heatmap.

- **Prediction time scales linearly with dataset size**, making KNN unsuitable for real-time chess evaluation where thousands of positions must be evaluated per second.
- **Model size is unbounded**: storing all training samples is infeasible as the dataset grows.
- **Generalization remains significantly weaker** than both Ridge and XGB, even after optimization.

For these reasons, despite completing hyperparameter optimization, the KNN model is excluded from further experimentation.

5.4. Final Remarks

Taken together, the hyperparameter optimization process validates the selection of two models for the final evaluation stage: **Ridge Regression** for the linear family and **XGBRegressor** for the tree-based family. The **KNeighborsRegressor**, while initially evaluated, was ultimately removed due to its computational constraints, scaling limitations, and inferior predictive performance.

6. Final Testing

6.1. Comparison of Optimized Models

Using the optimally tuned configurations obtained in the previous section, we evaluated both remaining candidate models—Ridge Regression and XGBRegressor—on the large, independently shuffled dataset `one_full_rd`. The results, summarized in Table 3, confirm the superiority of the gradient-boosted decision-tree model. While Ridge achieves competitive training speed, its predictive performance remains significantly below that of XGBRegressor.

The XGBRegressor preserves the high correlation levels observed during cross-validation despite being trained on a substantially

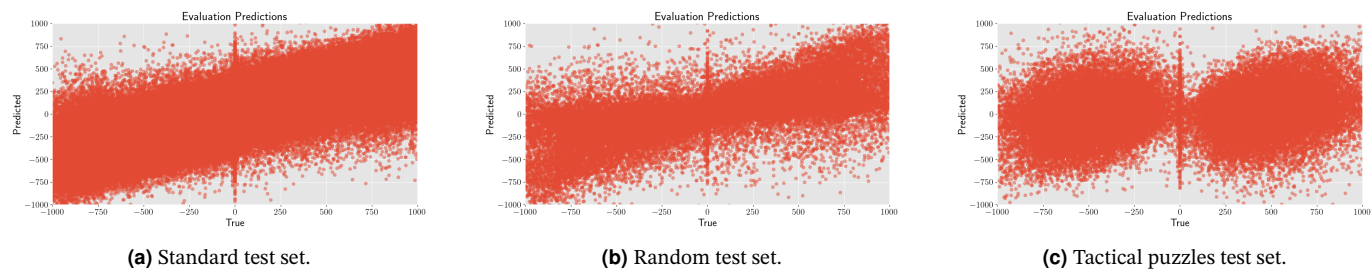


Figure 8. Comparison of model behavior across the three evaluation datasets.

Table 3. Performance comparison between the optimized models.

Model	Fit (s)	Prediction (s)	Spearman
XGBRegressor	557.97	20.23	0.6927
Ridge	2.05	0.20	0.5696

larger (and therefore more diverse) dataset. Although it requires significantly longer training, its superior accuracy justifies the computational cost. The reported prediction time corresponds to approximately 200 000 evaluations performed in batch for benchmarking; in a real engine, predictions are issued one at a time, making the per-position cost negligible.

6.2. Generalization Across Independent Datasets

To assess robustness beyond the training distribution, we evaluated the final model (trained on four million datapoints) on two additional datasets: (i) a fully independent random sample of engine evaluations (random_evals.csv), and (ii) a set of tactical puzzle positions (tactical_evals.csv) characterized by sharp, tactically loaded structures. In each case, we produced a scatter plot (Figure 8) of ground-truth evaluations against model predictions: the “standard” dataset (chessData.csv), the independent random sample, and the tactical dataset. These plots illustrate the varying degrees of structure the model is able to capture under different position distributions.

We report a complete set of evaluation metrics in Table 4.

Table 4. Performance of the final XGBRegressor model on three test sets.

Metric	Standard	Random	Tactical
Spearman	0.7032	0.6762	0.2012
Acc. (± 200)	0.8165	0.5582	0.1714
RMSE	190.65	352.90	542.89
Acc. (sign)	0.7181	0.7886	0.5518
Rec. (W)	0.8569	0.8643	0.6003
Rec. (B)	0.7201	0.7446	0.5163
R^2	0.5234	0.3913	-0.0536

The standard dataset exhibits the strongest alignment between predictions and true evaluations. The model achieves a Spearman coefficient in the range 0.65–0.70, high accuracy within a ± 200 centipawn window, and a positive R^2 . These values indicate that the model captures the structural dependencies of typical middlegame positions reasonably well.

Performance decreases moderately on the fully independent random dataset. The drop in accuracy and increase in RMSE indicate wider variance and noisier distributions, but the monotonic relationship remains strong (Spearman ≈ 0.68). This confirms that the model generalizes beyond the feature distribution of the training sample rather than relying on dataset-specific artifacts.

The tactical dataset exhibits a severe breakdown in predictive quality. Tactical puzzles depend on forcing sequences not captured by static evaluation. The very low Spearman correlation, large RMSE, and negative R^2 demonstrate that the model cannot infer the evaluation of sharp, non-positional positions without access to move-level or search-based information. This behavior aligns with expectations, as no feature in the model encodes tactical depth or forcing-line complexity.

7. Further Study

Future work includes integrating these models into a live chess engine to validate their effectiveness in real-time gameplay. This would test the hypothesis that learned evaluation functions can serve as substitutes for handcrafted heuristics, enabling engines to make competitive decisions with lower computational cost.

8. Code Usage

The code for reproducing the experiments and running the models is available on GitHub. To use it:

1. Load the chess_eval library:
- ```
1 from chess_eval import *
```
2. Load a model with its associated dataset and make the predictions:
- ```
1 model, dataset = load_model("
  xgb_four_full_rd")
2 model.predict(dataset.X_test)
```
3. Evaluate it:
- ```
1 evaluate(model, dataset)
```

This section allows readers to reproduce the results and explore alternative models. The full code and datasets are available online [1].

References

[1] Ferran Villarta and Natan Sisoiev. *Predicting Chess Position Evaluation from FEN*. <https://github.com/NatanSisoiev/Chess-ML-Evaluator>. 2025.

[2] *Chess Evaluations Dataset*. Accessed: 2025-11-12. URL: <https://www.kaggle.com/datasets/ronakbadhe/chess-evaluations/data>.

[3] *Forsyth–Edwards Notation*. Accessed: 2025-11-12. URL: [https://en.wikipedia.org/wiki/Forsyth%E2%80%93Edwards\\_Notation](https://en.wikipedia.org/wiki/Forsyth%E2%80%93Edwards_Notation).